Benchmarking local and commercial LLMs for political science text classification

Hanno Hilbig*

2026-05-06

Abstract

Local open-weight models are useful tools for social-science text classification: they avoid per-item API fees, keep data on the researcher's machine, and make exact model versions easier to preserve. I benchmark five local models against four commercial API models on 34 political science classification tasks. Local models are competitive, but task choice matters. The best local model matches or exceeds the best API model on 9 tasks, and the average best-local-minus-best-API gap is -0.015 headline F1. The four strongest models, Claude Sonnet 4.6, gpt-5.5, Gemma 4 31B, and DeepSeek V4 Pro, fall within 0.021 mean headline F1. API models have their clearest edge on complex tasks with many labels or multiple outputs. Batching several texts in one prompt improves local throughput, but some task-model combinations return invalid labels or invalid response formats. The practical recommendation is simple: benchmark candidate models on ground-truth labels for the target task, report both performance and reliability, and test batching before using it at scale.

1 Motivation

When using LLMs to classify text data, researchers can use commercial API models or local open-weight models. Commercial API models are typically reliable and easy to call from R or Python. However, they cost money per item, send data off-machine, and create reproducibility risk when model behavior changes over time. Local open-weight models are free to run, keep the data on the researcher's machine, and make exact model versions easier to preserve and track. This matters, for example, when data is restricted or cannot be sent to third-party APIs for other reasons.

I run a benchmark of 34 classification tasks to assess the performance of local models across a range of applications. I am particularly interested in three questions: (i) when local models approach or exceed commercial API performance, (ii) how much local inference time researchers should expect, and (iii) when task complexity widens the local/API gap.

2 Design

The benchmark has two parts. First, I run a single-item prompt benchmark covering thirty-four classification tasks drawn from published political science replication archives and nine models: five local open-weight models and four commercial API models from OpenAI, Anthropic, and DeepSeek. Each model sees the same items within a task. Thirty-one tasks use 500 sampled items. Three tasks have fewer cleaned items available, so I use all available observations: 320 for Brandt political relevance, 312 for PLOVER CAMEO event coding, and 293 for COVID threat minimization. This is why task sizes range from 293 to 500 observations.

I report performance and reliability for each task and model. The headline performance measure is F1, defined to match the task: positive-class F1 for binary tasks, mean per-label F1 for multi-binary tasks, and macro F1 for categorical tasks. Accuracy and Matthews correlation coefficient (MCC) provide additional

*hhilbig@ucdavis.edu. Code and data: https://github.com/hhilbig/polsci-open-bench.

measures of overall agreement. I also record wall-clock latency per item and the share of unusable responses. A response is unusable if it fails to parse, returns the wrong object shape, or uses a label outside the task schema. Performance metrics are computed over usable outputs only, so failure rates should be read alongside F1, accuracy, and MCC.

I ran each model on each item once in the single-item benchmark. The local setup was a single Apple Silicon MacBook with 32 GB of unified memory running Ollama at temperature 0.1. Commercial API calls used provider-specific constrained-output settings. OpenAI tiers used `reasoning_effort=medium`; DeepSeek ran with internal thinking disabled; OpenAI and DeepSeek returned JSON-constrained outputs; and Anthropic used tool use to constrain labels. Prompts are verbatim from the source paper's coding instructions where possible; otherwise, I derive them from the published codebook (see Table 1).

Second, I test whether prompt batching speeds up local inference. In the b=10 batching experiment, the model receives ten texts in one prompt and returns ten labels in a single response. The point is to reduce per-item latency without changing the classification task. The local b=10 grid covers all thirty-four tasks and all five local models.

Within-task comparisons use 95% paired-by-item bootstrap confidence intervals (1000 iterations). I treat the thirty-four tasks as a fixed benchmark population, but also report paired task-level bootstrap intervals that resample tasks with replacement.

3 Results

3.1 Average performance

Figure 1 shows the 34-task model averages and the spread of task-level performance within each model. Claude Sonnet 4.6 has the highest observed mean headline F1 (0.668), followed by gpt-5.5 (0.660), Gemma 4 31B (0.648), and DeepSeek V4 Pro (0.647). The spread across these four models is 0.021 points. The same broad top group appears under mean accuracy and mean MCC (Table 5), although the exact order changes slightly. Among the OpenAI tiers, gpt-5.4-nano remains the cost-efficient option, at 0.632 mean headline F1 compared with 0.660 for gpt-5.5.

The task-level bootstrap corroborates this interpretation (Table 3). The Claude Sonnet 4.6 versus gpt-5.5 interval includes zero, Gemma 4 31B and DeepSeek V4 Pro are effectively tied, and the estimated Claude advantage over Gemma 4 31B and DeepSeek V4 Pro is about two headline F1 points. Many within-task leader differences are also too uncertain to treat as firm rankings. The practical point is that small average gaps do not justify a stable leaderboard.

The local/API comparison in Figure 2 is the most direct test of my question. After selecting the best model within each class for each task, the best local model matches or exceeds the best API model on 9 of the 34 tasks. The mean best-local-minus-best-API gap is -0.015 headline F1, and the median gap is -0.019. Figure 2 plots the same comparison as best API minus best local, so values to the left of zero favor local models. This is not evidence that local models dominate API models. It shows that the API advantage is often small enough that local inference is worth validating on the target task.

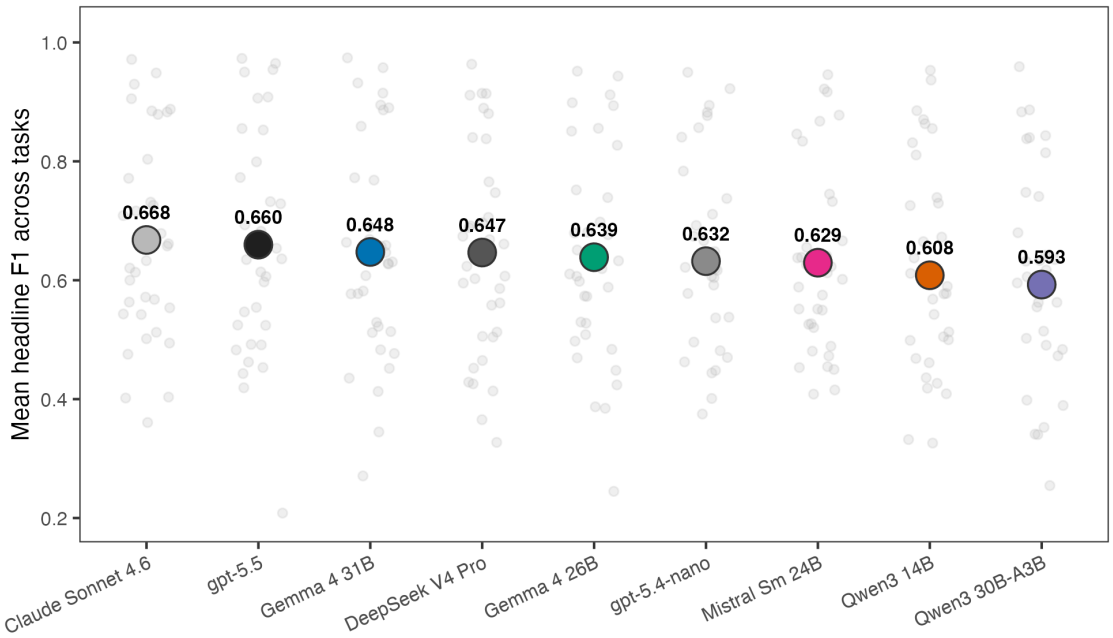

Figure 1: Mean headline F1 across thirty-four tasks per model. Large filled circles show 34-task means. Small gray dots show observed per-task headline F1 values.

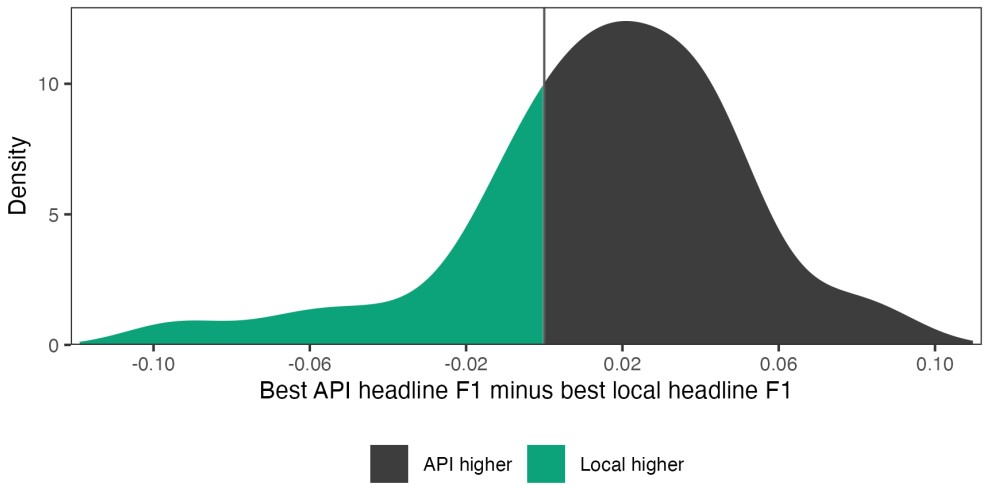

Figure 2: Density of best API minus best local headline F1 across tasks. Negative values favor the local model class; positive values favor the API model class. The green area shows the local-favoring side of the distribution; the black area shows the API-favoring side.

The model selection upper bounds in Figure 7 lead to the same conclusion. These upper bounds choose the best model after observing performance on each task. Allowing the local model to vary by task raises mean headline F1 from 0.648 for the best single local model to 0.673. Across all nine models, task-specific selection reaches 0.696, compared with 0.668 for the best single model. These values show the upside of task-specific validation, but not how large a validation sample is needed for reliable model selection.

The larger differences are across tasks. The paired-by-item intervals are wide enough that many task-level leader differences remain uncertain. The benchmark should therefore not be read primarily as a leaderboard:

global rank is a weak decision rule for applied coding.

3.2 Models are complementary across tasks

Figure 3 groups the thirty-four tasks into five broad annotation types: relevance and harm, position and tone, events and actions, claims and relations, and issues and topics. These are report groupings, not source-provided task families, and borderline cases are assigned by prediction target. The type-level averages do not show a simple local versus API pattern.

At the task level, the top point estimates are split across eight models. gpt-5.5 leads on thirteen tasks, Claude Sonnet 4.6 leads on eight, Mistral Small leads on three, and five other models lead on two tasks each.

The winner split matters because it suggests complementarity across models. Even models with lower average performance sometimes produce the best task-level point estimate. The models are not simply ordered by a single general-capability scale.

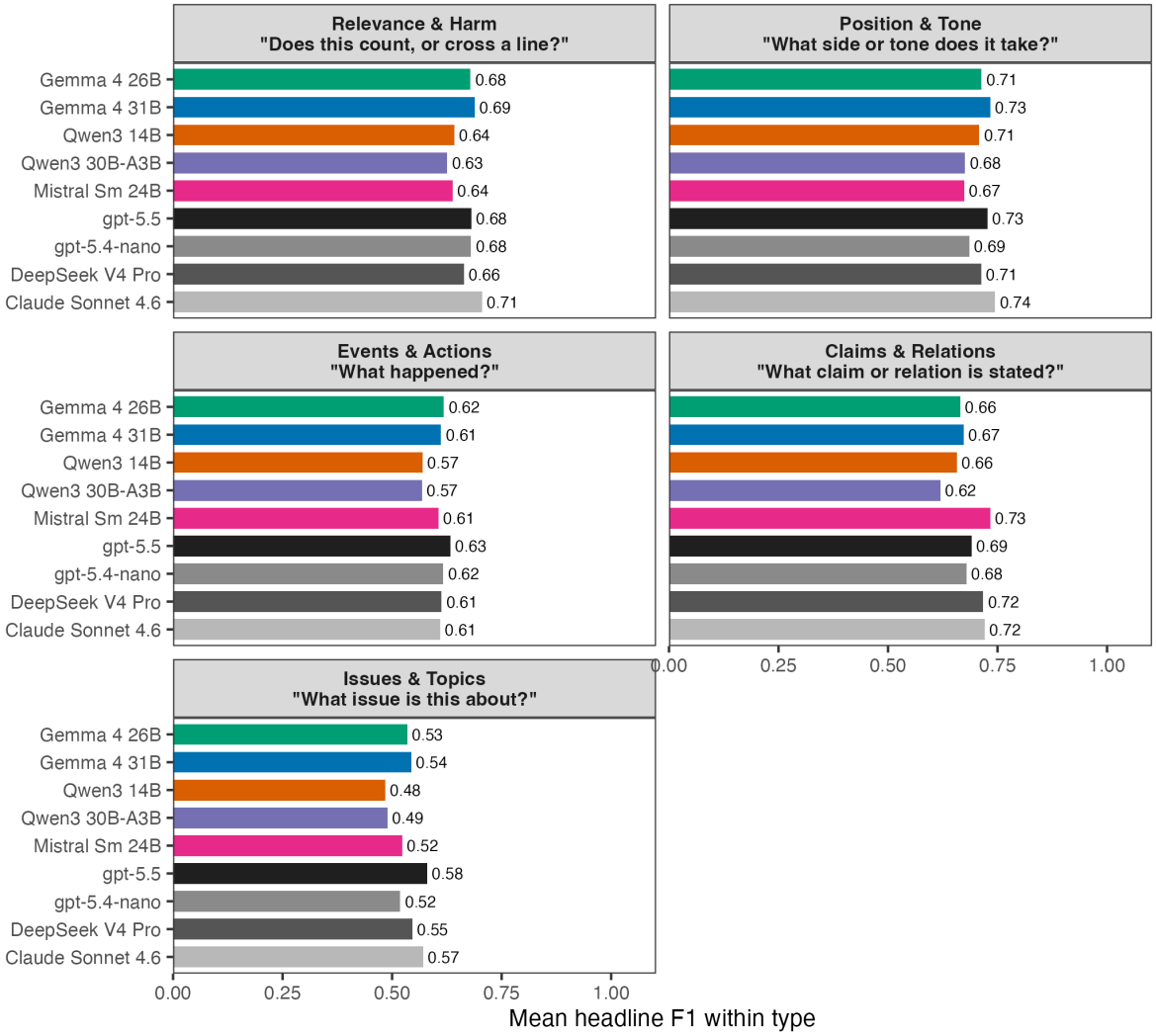

Figure 3: Mean headline F1 within five broad annotation types, by model. Type-level averages are descriptive only.

3.3 Performance is lower on more complex coding tasks

Figure 4 asks whether coding complexity helps explain the large task-to-task variation. I measure complexity with two features that researchers can observe before running a benchmark: the prompt/codebook and the number of labels used in the gold data. Low-complexity tasks have fewer than three effective labels and short prompts. Medium-complexity tasks have either more labels or longer prompts. High-complexity tasks have at least eight effective labels or require multi-binary outputs.

The pattern is clear, but the gaps are not large enough to erase local competitiveness. Mean headline F1 falls from 0.70 to 0.57 for API model-task cells and from 0.67 to 0.52 for local cells as tasks move from low to high complexity. When I instead compare the best API model with the best local model on each task, the low-complexity gap nearly disappears (0.740 versus 0.735), while the high-complexity gap is about 0.05 headline F1 (0.605 versus 0.555). This is the clearest API edge in the complexity bins. It points to the tasks where validation matters most: long codebooks, many effective labels, and multi-output schemas.

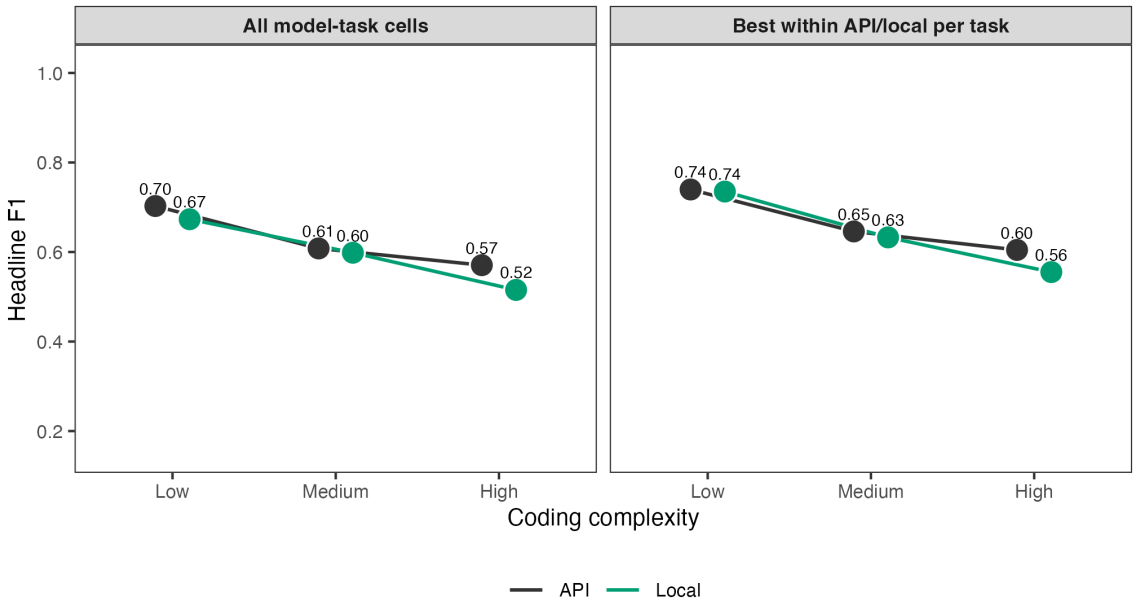

Figure 4: Coding complexity and model performance across the 34-task benchmark. The left panel shows mean headline F1 across all observed task-model combinations, separately for local and API models. The right panel shows the best local model and best API model per task within each complexity group. Points show group means; lines connect complexity groups within model class.

3.4 Local inference is not uniformly slower

Local inference speed varies sharply with model size, architecture, and batching (Figure 5). On a typical 500-item task, median single-item runtimes range from about 5 minutes for Qwen3-30B-A3B and 7 minutes for Gemma 4 26B to about 29 minutes for Gemma 4 31B. Qwen3-30B-A3B is fastest because it is a sparse model that uses about three billion active parameters per call. Gemma 4 31B has the strongest local 34-task single-item headline F1, while Gemma 4 26B is much faster and remains close on average. The point is practical: local inference is slow enough to plan around, but not slow enough to rule out ordinary validation and production coding.

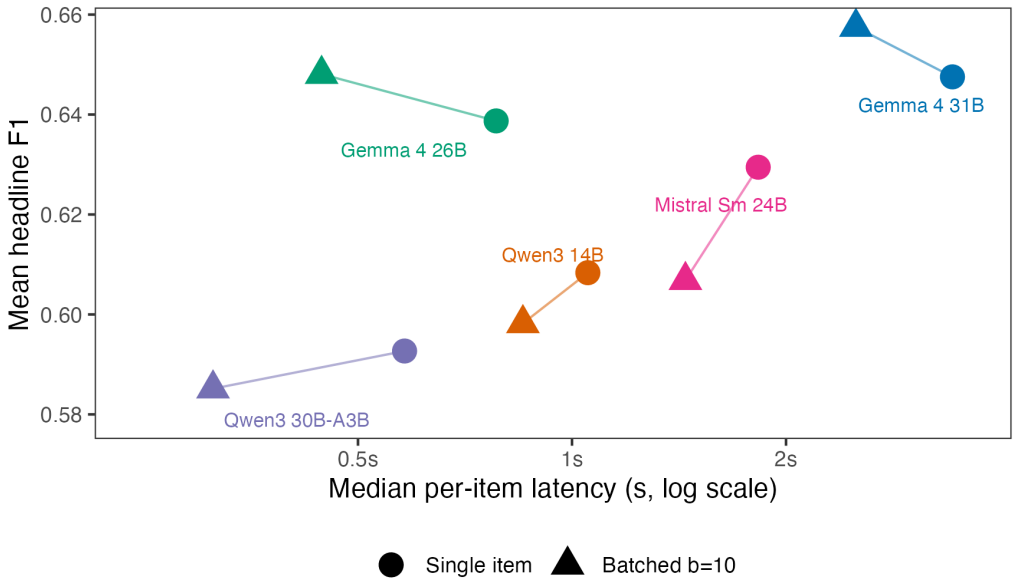

Figure 5: Mean headline F1 vs median per-item latency, for the five local models. Circles show single-item calls over the 34-task benchmark; triangles show b=10 batched calls over the same task set. Lines connect the two call modes for the same model, and labels identify model pairs. API models are excluded because their per-call wall-clock mixes server load, queueing, and network round-trip and is not directly comparable to local inference time.

All five local models ran on a single 32 GB Apple Silicon MacBook; the largest, Gemma 4 31B, fit in unified memory. Appendix Figure 6 translates the local latencies into median minutes per 1,000 observations for each model. For reference, the API tiers in this run sat between roughly 0.9 and 1.7 s/item on median latency end-to-end, but those numbers mix model speed with provider serving load, network, and queueing and are not shown in the figure.

3.5 Batching improves throughput but exposes reliability failures

Prompt batching changes the prompt: the model receives several texts at once and must return one label for each text. Provider Batch API mode is different. It changes how requests are queued and billed, but each text still appears in a separate request. The procedure I test here is prompt batching. Pipal et al. (2026) show why this matters for annotation workflows: one-at-a-time classification leaves much of the available throughput unused.

Figure 5 shows the main local result. Prompt batching reduces median per-item latency for every local model, with model-level median speedups from about 1.23× to 1.86×. Average headline-F1 changes are modest, ranging from -0.023 to +0.010. Gemma 4 26B is the most useful fast batched option in these results: it remains close to Gemma 4 31B on average but classifies a typical 500-item task in about 4 minutes at b=10, compared with about 21 minutes for Gemma 4 31B.

Prompt length still matters. Short-prompt cells have a median local b=10 speedup of about 1.34×, while long-codebook cells have a median speedup of about 1.20×. Schema and label reliability is the sharper constraint. These failures include outputs that do not parse, outputs with the wrong object or array shape, and outputs that parse but return labels outside the permitted task schema. The largest failures are concentrated in specific model-task pairs. For example, Gemma 4 26B produces unusable responses for 72% of Erlich ATI items at b=10, Qwen3-30B-A3B produces unusable responses for 51% of Halterman CCC items at b=10, and gpt-5.5 returns unusable output for 68% of Halterman CCC items in the limited API prompt-batching run. The appendix table on schema and label failure rates lists the covered cells above 5%.

3.6 Cost

The OpenAI portion of the 34-task single-item benchmark remains cheap enough for validation-scale benchmarking, but the tiers differ sharply. At publicly listed prices, gpt-5.4-nano costs about $4 for the 16,425 predictions in the 34-task benchmark, while gpt-5.5 would cost about $71 under direct per-request pricing. In return, gpt-5.5 improves mean headline F1 from 0.632 to 0.660. The smaller tier is therefore worth screening before moving to a flagship API model.

Provider Batch API mode changes the cost calculation for flagship models. The gpt-5.5 runs for a 16-task subset used the OpenAI Batch API: 7,605 requests completed with 0 parse/API errors at an estimated batch-discounted cost of $14.13 under a $20 budget cap. Claude Sonnet 4.6 was also run through provider Batch API mode for the same subset, with 7,604 of 7,605 outputs parsing cleanly after retries. Local models avoid API charges, but the relevant cost becomes wall-clock time, hardware availability, and energy use.

Implications for applied use

The results imply a workflow rather than a single best model. Start with a small validation sample on the target task: the benchmark average does not reliably predict which model performs best on a specific task. All five local models fit on a 32 GB Apple Silicon MacBook, so one laptop setup is enough hardware for the full benchmark.

Recommended workflow. Start with 100 to 300 hand-labeled validation cases. Test at least one cheap API model, one flagship API model, and two useful local models. Report headline F1, accuracy, MCC, and schema or label failures. For imbalanced tasks, inspect rare-class recall. Use batching only after checking task-level schema reliability, and freeze the exact model version before production coding.

For metric reporting on imbalanced label sets, headline F1, accuracy, and MCC should appear together because each answers a different question. Mellon BES MII, a heavily imbalanced issue-classification task, illustrates the point: for the top model, headline F1 is about 0.55, accuracy is 0.95, and MCC is 0.93. When paid API use is acceptable, cheap tiers should also be screened rather than dismissed: in this run, gpt-5.4-nano lost 0.028 mean headline F1 relative to gpt-5.5 but cost about $4 for the full 34-task benchmark.

For batching, b=10 is a reasonable starting point for local models when throughput matters. The average performance cost is small, but schema and label reliability must be checked at the task-model level. For long, multi-class codebooks, researchers should not batch aggressively unless retries are built in.

Limitations and scope

Model selection and prompting. The model set reflects five specific local models that fit a 32 GB Apple Silicon setup, plus four commercial API tiers that include cheaper and flagship options. I did not optimize prompts separately per model. Some are verbatim from the source paper, others are derived from published codebooks.

Beyond this benchmark. Local inference keeps text on the researcher's machine, which can simplify workflows with sensitive or restricted data. The benchmark studies prompt-based classification only: for large labeled datasets with fixed label sets, fine-tuned local models, supervised classifiers, or embedding-based methods may be cheaper and more reliable. API endpoints can change behavior between releases even when the model name is stable, while local open-weight models are easier to freeze exactly.

References

Ballard, Andrew O., Ryan DeTamble, Spencer Dorsey, Michael Heseltine, and Marcus Johnson. 2022. "Incivility in Congressional Tweets." *American Politics Research* 50 (6): 769–80. https://doi.org/10.1177/1532673X221109516.

Bestvater, Samuel E., and Burt L. Monroe. 2023. "Sentiment Is Not Stance: Target-Aware Opinion Classification for Political Text Analysis." *Political Analysis* 31 (2): 235–56. https://doi.org/10.1017/pan.2022.10.

Brandt, Patrick T., Sultan Alsarra, Vito D'Orazio, et al. 2025. "Extractive Versus Generative Language Models for Political Conflict Text Classification." *Political Analysis*, ahead of print. https://doi.org/10.1017/pan.2025.10027.

Burnham, Michael. 2025. "Stance Detection: A Practical Guide to Classifying Political Beliefs in Text." *Political Science Research and Methods* 13 (3): 611–28. https://doi.org/10.1017/psrm.2024.35.

Burnham, Michael, Kayla Kahn, Ryan Yang Wang, and Rachel X. Peng. 2025. "Political DEBATE: Efficient Zero-Shot and Few-Shot Classifiers for Political Text." *Political Analysis*, 1–15. https://doi.org/10.1017/pan.2025.10028.

Chae, Youngjin, and Thomas Davidson. 2025. "Large Language Models for Text Classification: From Zero-Shot Learning to Instruction-Tuning." *Sociological Methods & Research*, ahead of print. https://doi.org/10.1177/00491241251325243.

Comparative Agendas Project. 2019. *Master CAP Codebook*. https://www.comparativeagendas.net/pages/master-codebook.

Congressional Research Service. 2026. *CRS Reports*. https://www.congress.gov/crs-products.

Di Cocco, Jessica, and Bernardo Monechi. 2022. "How Populist Are Parties? Measuring Degrees of Populism in Party Manifestos Using Supervised Machine Learning." *Political Analysis* 30 (3): 311–27. https://doi.org/10.1017/pan.2021.29.

Douglass, Rex W., Thomas Leo Scherer, J. Andrés Gannon, et al. 2024. "Introducing ICBe: An Event Extraction Dataset from Narratives about International Crises." *Political Science Research and Methods* 12 (4): 729–49. https://doi.org/10.1017/psrm.2024.17.

Erlich, Aaron, Stefano G. Dantas, Benjamin E. Bagozzi, Daniel Berliner, and Brian Palmer-Rubin. 2022. "Multi-Label Prediction for Political Text-as-Data." *Political Analysis* 30 (4): 463–80. https://doi.org/10.1017/pan.2021.15.

Garcia-Corral, Paulina, Hannah Béchara, Ran Zhang, and Slava Jankin. 2024. "PolitiCause: An Annotation Scheme and Corpus for Causality in Political Texts." *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, 12836–45. https://aclanthology.org/2024.lrec-main.1124/.

Gilardi, Fabrizio, Meysam Alizadeh, and Maël Kubli. 2023. "ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks." *Proceedings of the National Academy of Sciences* 120 (30): e2305016120. https://doi.org/10.1073/pnas.2305016120.

González-Bustamante, Bastián. 2024. *Benchmarking LLMs in Political Content Text-Annotation: Proof-of-Concept with Toxicity and Incivility Data*. https://arxiv.org/abs/2409.09741.

Halterman, Andrew, Benjamin E. Bagozzi, Andreas Beger, Philip A. Schrodt, and Grace I. Scarborough. 2023. *Plover and Polecat: A New Political Event Ontology and Dataset*. https://www.andrewhalterman.com/publication/plover-polecat-new-event-data/.

Halterman, Andrew, and Katherine A. Keith. 2025. "Codebook LLMs: Evaluating LLMs as Measurement Tools for Political Science Concepts." *Political Analysis*, 1–17. https://doi.org/10.1017/pan.2025.10017.

Haunss, Sebastian, Priska Daphi, Jan Matti Dollbaum, Lidiya Hristova, Pál Susánszky, and Elias Steinhilper. 2025. "PAPEA: A Modular Pipeline for the Automation of Protest Event Analysis." *Political Science Research and Methods*, 1–18. https://doi.org/10.1017/psrm.2025.10013.

Mellon, Jonathan, Jack Bailey, Ralph Scott, James Breckwoldt, Marta Miori, and Phillip Schmedeman. 2024. "Do AIs Know What the Most Important Issue Is? Using Language Models to Code Open-Text Social Survey Responses at Scale." *Research & Politics* 11 (1): 1–7. https://doi.org/10.1177/20531680241231468.

Müller, Stefan, and Naofumi Fujimura. 2025. "Campaign Communication and Legislative Leadership." *Political Science Research and Methods* 13: 545–66. https://doi.org/10.1017/psrm.2024.11.

Ornstein, Joseph T., Elise N. Blasingame, and Jake S. Truscott. 2025. "How to Train Your Stochastic Parrot: Large Language Models for Political Texts." *Political Science Research and Methods* 13 (2): 264–81. https://doi.org/10.1017/psrm.2024.64.

Osnabrügge, Moritz, Elliott Ash, and Massimo Morelli. 2023. "Cross-Domain Topic Classification for Political Texts." *Political Analysis* 31 (1): 78–91. https://doi.org/10.1017/pan.2021.37.

Pendzel, Sagi, Nir Lotan, Alon Zoizner, and Einat Minkov. 2023. *Detecting Multidimensional Political Incivility on Social Media*. https://arxiv.org/abs/2305.14964.

Pipal, Christian, Eva-Maria Vogel, Morgan Wack, and Frank Esser. 2026. *Researchers Waste 80% of LLM Annotation Costs by Classifying One Text at a Time*. https://arxiv.org/abs/2604.03684.

Rheault, Ludovic, Erica Rayment, and Andreea Musulan. 2019. "Politicians in the Line of Fire: Incivility and the Treatment of Women on Social Media." *Research & Politics* 6 (1). https://doi.org/10.1177/2053168018816228.

Sermpezis, Pavlos, Stelios Karamanidis, Eva Paraschou, et al. 2024. *AgoraSpeech: A Multi-Annotated Comprehensive Dataset of Political Discourse Through the Lens of Humans and AI*. https://doi.org/10.5281/zenodo.13957177.

Theocharis, Yannis, Pablo Barberá, Zoltán Fazekas, and Sebastian Adrian Popa. 2020. "The Dynamics of Political Incivility on Twitter." *SAGE Open* 10 (2). https://doi.org/10.1177/2158244020919447.

Zhang, Meiqing, Furkan Cakmak, Markus Neumann, et al. 2025. "Comparable 2022 General Election Advertising Datasets from Meta and Google." *Scientific Data* 12: 968. https://doi.org/10.1038/s41597-025-05228-w.

Companion materials

Prompt provenance, batched inference details, summary CSVs, and reproduction instructions are on the GitHub repository at https://github.com/hhilbig/polsci-open-bench.

Tasks

Table 1: Thirty-four tasks in the benchmark. 'Type' is the report grouping used in Figure 3; it is a broad annotation type rather than a source-provided task family. 'Provenance': Verbatim = prompt unchanged from source paper, Verbatim-adapted = source content with format-only changes, Derived = our prompt reasoned from the published codebook. 'Prompt': short (15-35 lines) or long codebook (47-126 lines). This is the trait that determines local batching feasibility. The CMP task uses the Halterman & Keith (2025) domain-collapsed 7-class version of the CMP/MARPOR coding scheme, not the full CMP category set.

Task	Description	Source	Type	Labels	Provenance	Prompt
Gilardi relevance	Tweets, classified as about content moderation or not.	Gilardi 2023	Relevance & Harm	binary	Verbatim	short
Brandt relevance	News articles, classified as politics-relevant or not.	Brandt et al. 2025	Relevance & Harm	binary	Derived	short
Burnham PolNLI	Premise-hypothesis pairs, classified for entailment.	Burnham et al. 2025	Claims & Relations	binary	Derived	short
Ballard incivility	Tweets by U.S. members of Congress, classified for incivility.	Ballard 2022	Relevance & Harm	binary	Derived	short
Line of Fire incivility	Political tweets, classified for incivility.	Rheault et al. 2019	Relevance & Harm	binary	Derived	short
Theocharis incivility	Political tweets, classified for incivility.	Theocharis et al. 2020	Relevance & Harm	binary	Derived	short
Gilardi stance	Tweets about content moderation, classified by stance toward moderation.	Gilardi 2023	Position & Tone	3-class	Verbatim	short
SemEval stance	Tweets toward five named political targets, classified by stance.	Chae & Davidson 2025	Position & Tone	3-class	Derived	short
SCOTUS sentiment	Tweets about U.S. Supreme Court rulings, classified by sentiment.	Ornstein et al. 2025	Position & Tone	3-class	Derived	short
Wesleyan ad tone	U.S. Meta political ads, classified by tone toward a candidate.	Zhang et al. 2025	Position & Tone	3-class	Derived	short
CCC protest type	News stories about U.S. protest events, classified by event type.	Halterman & Keith 2025	Events & Actions	4-class	Verbatim-adapted	long codebook
BFRS violence	News stories about Pakistani political violence, classified by event type.	Halterman & Keith 2025	Events & Actions	12-class	Verbatim-adapted	long codebook
PAPEA protest forms	Protest snippets, classified by protest form.	Haunss et al. 2025	Events & Actions	7-class	Derived	long codebook
ICBe event type	Crisis-event sentences, classified by sentence event type.	Douglass et al. 2024	Events & Actions	5-class	Derived	short
CMP manifestos	Quasi-sentences from political party manifestos, classified by policy domain.	Halterman & Keith 2025	Issues & Topics	7-class	Derived	long codebook
Cross-domain topics	Political text, classified by broad policy domain.	Osnabruegge et al. 2023	Issues & Topics	8-class	Derived	long codebook
Campaign policy area	Campaign statements, classified by policy area.	Muller & Fujimura 2024	Issues & Topics	12-class	Derived	long codebook

Table 1: Thirty-four tasks in the benchmark. 'Type' is the report grouping used in Figure 3; it is a broad annotation type rather than a source-provided task family. 'Provenance': Verbatim = prompt unchanged from source paper, Verbatim-adapted = source content with format-only changes, Derived = our prompt reasoned from the published codebook. 'Prompt': short (15-35 lines) or long codebook (47-126 lines). This is the trait that determines local batching feasibility. The CMP task uses the Halterman & Keith (2025) domain-collapsed 7-class version of the CMP/MARPOR coding scheme, not the full CMP category set.

Task	Description	Source	Type	Labels	Provenance	Prompt
BES MII	Open-ended British Election Study responses, classified by issue topic.	Mellon et al. 2024	Issues & Topics	50-class	Verbatim-adapted	long codebook
Spanish protest toxicity	Spanish protest tweets, classified for toxic content.	toxicity-protests-ES	Relevance & Harm	binary	Derived	short
TwitCivility impoliteness	Political tweets, classified for impoliteness.	TwitCivility	Relevance & Harm	binary	Derived	short
GTD attack type	Global Terrorism Database events, classified by primary attack type.	Brandt et al. 2025	Events & Actions	9-class	Derived	long codebook
PAPEA protest claims	Protest snippets, classified by protest claim.	Haunss et al. 2025	Issues & Topics	28-class	Derived	long codebook
PLOVER CAMEO event	Gold-standard event records, classified by CAMEO event type.	PLOVER	Events & Actions	18-class	Derived	long codebook
PolNLI event entailment	Event-extraction PolNLI pairs, classified for entailment.	Burnham et al. 2025	Claims & Relations	binary	Derived	short
Women's March stance	Tweets about the Women's March, classified by stance.	Bestvater & Monroe	Position & Tone	binary	Derived	short
Trump stance	Political statements, classified by stance toward Trump.	Burnham et al. 2025	Position & Tone	3-class	Derived	short
COVID threat minimization	Political text, classified for COVID threat minimization.	Burnham et al. 2025	Claims & Relations	binary	Derived	short
Kavanaugh stance	Tweets about Brett Kavanaugh, classified by stance.	Bestvater & Monroe	Position & Tone	binary	Derived	short
ATI topics	Mexican access-to-information requests, classified by request topic.	Erlich et al.	Issues & Topics	7-label	Derived	long codebook
CAP party platforms	Party-platform quasi-statements, classified by CAP major topic.	CAP	Issues & Topics	21-class	Derived	long codebook
CAP CRS reports	Congressional Research Service titles and summaries, classified by CAP major topic.	CAP / CRS	Issues & Topics	21-class	Derived	long codebook
PolitiCAUSE causal relation	Political sentences, classified for whether they express a causal relation.	PolitiCAUSE	Claims & Relations	binary	Derived	short
Manifesto populism	Italian manifesto sentences, classified for populist rhetoric.	Di Cocco & Monechi	Claims & Relations	binary	Derived	short
AgoraSpeech criticism/agenda	Greek campaign-speech paragraphs, classified as criticism or agenda setting.	AgoraSpeech	Claims & Relations	2-class	Derived	short

Per-task winners

Table 2: Top model per task by headline F1 across the unified 34-task benchmark, grouped by the report's five annotation types. Accuracy and MCC refer to the same model-task cell. CIs and the full per-(task, model) table are on the GitHub repository.

Type	Task	Top model	Headline F1	Accuracy	MCC
Relevance & Harm	Ballard incivility	Claude Sonnet 4.6	0.563	0.848	0.515
	Brandt relevance	Qwen3 14B	0.739	0.887	0.703
	Gilardi relevance	gpt-5.5	0.950	0.954	0.908
	Line of Fire incivility	DeepSeek V4 Pro	0.607	0.886	0.561
	Spanish protest toxicity	Gemma 4 26B	0.894	0.892	0.784
	Theocharis incivility	gpt-5.5	0.682	0.812	0.551
	TwitCivility impoliteness	Gemma 4 31B	0.631	0.794	0.503
Position & Tone	Gilardi stance	Claude Sonnet 4.6	0.568	0.648	0.471
	Kavanaugh stance	gpt-5.5	0.954	0.952	0.906
	SCOTUS sentiment	DeepSeek V4 Pro	0.668	0.718	0.543
	SemEval 2016 stance	gpt-5.5	0.773	0.768	0.659
	Trump stance	Claude Sonnet 4.6	0.804	0.826	0.730
	Wesleyan ad tone	gpt-5.4-nano	0.711	0.918	0.808
	Women's March stance	Claude Sonnet 4.6	0.905	0.852	0.623
Events & Actions	BFRS violence (12-cl)	gpt-5.5	0.693	0.730	0.691
	CCC protest type	Qwen3 14B	0.499	0.610	0.387
	GTD attack type (9-cl)	gpt-5.4-nano	0.617	0.750	0.656
	ICBe event type (5-cl)	Gemma 4 26B	0.530	0.584	0.463
	PAPEA protest forms (7-cl)	gpt-5.5	0.965	0.972	0.958
	PLOVER CAMEO event (18-cl)	gpt-5.5	0.654	0.699	0.682
Claims & Relations	AgoraSpeech criticism/agenda	Mistral Sm 24B	0.922	0.924	0.843
	Burnham PolNLI	gpt-5.5	0.908	0.928	0.853
	COVID threat minimization	Mistral Sm 24B	0.602	0.819	0.499
	Manifesto populism	Mistral Sm 24B	0.455	0.904	0.403
	PolNLI event entailment	Gemma 4 31B	0.974	0.972	0.945
	PolitiCAUSE causal relation	Claude Sonnet 4.6	0.716	0.772	0.578
Issues & Topics	ATI topics (7-label)	Claude Sonnet 4.6	0.502	NA	NA
	BES MII (50-cl)	gpt-5.5	0.547	0.946	0.934
	CAP CRS reports (21-cl)	gpt-5.5	0.732	0.774	0.757
	CAP party platforms (21-cl)	Claude Sonnet 4.6	0.731	0.736	0.719
	CMP manifestos (7-cl)	gpt-5.5	0.597	0.646	0.545
	Campaign policy area (12-cl)	gpt-5.5	0.636	0.750	0.691
	Cross-domain topics (8-cl)	Claude Sonnet 4.6	0.476	0.494	0.433
	PAPEA protest claims (28-cl)	gpt-5.5	0.462	0.636	0.606

Task-level uncertainty for model averages

Table 3: Paired task-level bootstrap intervals for cross-task mean headline-F1 differences among the four strongest models. Each bootstrap resamples the 34 tasks with replacement and recomputes the paired mean difference. These intervals assess sensitivity to benchmark composition; they do not replace the paired-by-item bootstrap intervals used for within-task comparisons.

Model comparison	Mean difference	95% task-bootstrap interval	Tasks
Claude Sonnet 4.6 - gpt-5.5	0.008	[-0.008, 0.024]	34
Claude Sonnet 4.6 - Gemma 4 31B	0.020	[0.005, 0.038]	34
Claude Sonnet 4.6 - DeepSeek V4 Pro	0.021	[0.004, 0.040]	34
gpt-5.5 - Gemma 4 31B	0.012	[-0.007, 0.033]	34
gpt-5.5 - DeepSeek V4 Pro	0.013	[-0.006, 0.032]	34
Gemma 4 31B - DeepSeek V4 Pro	0.001	[-0.020, 0.020]	34

Local runtime per 1,000 observations

Figure 6 converts local median per-item latency into minutes per 1,000 observations. The b=10 bars use the full 34-task local batching grid, so they should be read alongside the schema and label reliability checks below.

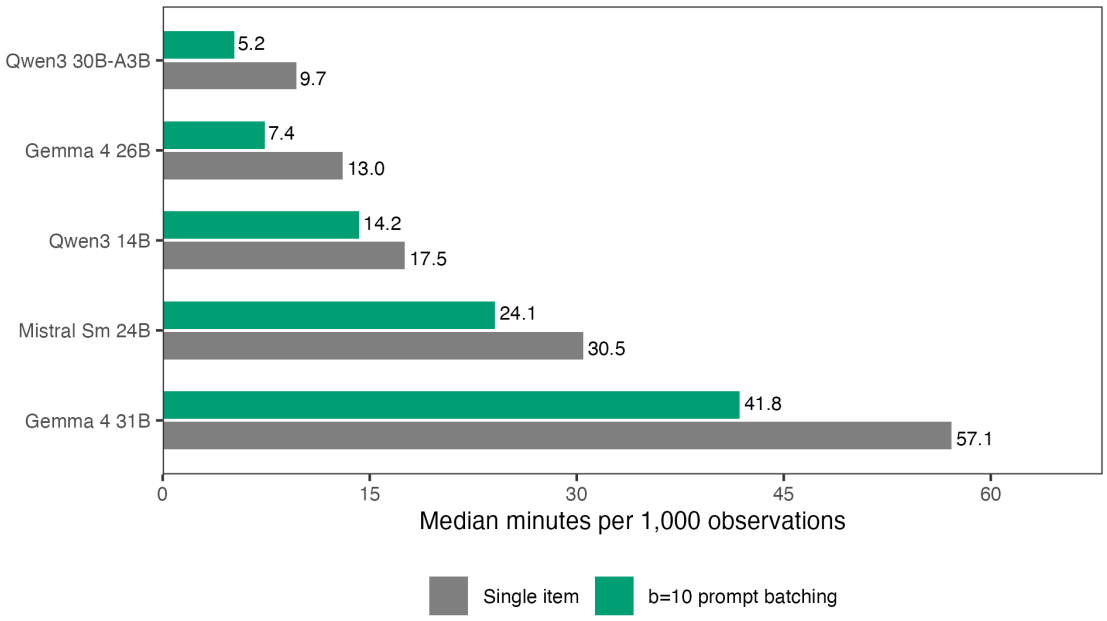

Figure 6: Median local runtime per 1,000 observations. Single-item and b=10 bars use the 34-task benchmark. Lower values are faster. Batching should be interpreted alongside schema/label reliability checks.

Additional model-selection and label-structure checks

Figure 2 in the main text compares the best local model with the best API model separately for each task. The largest API advantages appear on tasks that involve policy domains, event codes, causal relations, or protest claims. The largest local advantages appear on more focused tasks such as manifesto populism, COVID threat minimization, and CCC protest type.

Figure 7 summarizes the value of task-specific model selection. The three single-model bars use one model for all thirty-four tasks. The three oracle bars select the best model within each task and then average across tasks. These oracle quantities are descriptive upper bounds, not deployable estimates, because they choose the model after observing task performance.

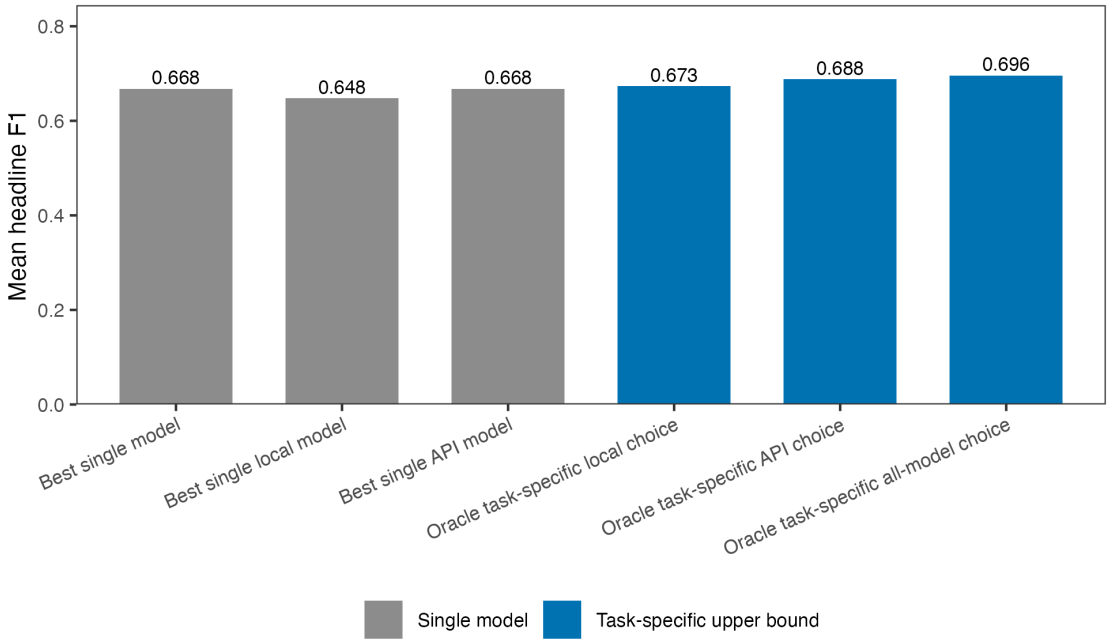

Figure 7: Mean headline F1 under single-model and task-specific selection rules. Oracle task-specific rules are descriptive upper bounds that select the best model after observing task-level performance.

The best single model is Claude Sonnet 4.6, with a 34-task mean headline F1 of 0.668. The best single local model, Gemma 4 31B, reaches 0.648. If the local model is allowed to vary by task, the local oracle rises to 0.673. The API oracle reaches 0.688, and the all-model oracle reaches 0.696. Because the oracle rules choose after observing outcomes, these numbers are upper bounds on what validation can buy, not expected production performance.

Figure 8 relates the local/API task gap to the effective number of labels in the sampled gold data. Effective label count captures how many labels actually matter in the sample, rather than simply counting all possible labels. A balanced four-class task therefore has a higher effective count than a four-class task where almost all observations use the same label. The figure is exploratory, but it ties the complexity result to one observable feature of the task: how many labels are actively used.

The relationship is negative: tasks with more effectively used labels tend to show larger API advantages. The simple correlation between effective label count and the best-local-minus-best-API gap is about -0.50. Binary and two-class tasks have a mean gap of -0.004, while many-class or multi-label tasks have a mean gap of -0.025.

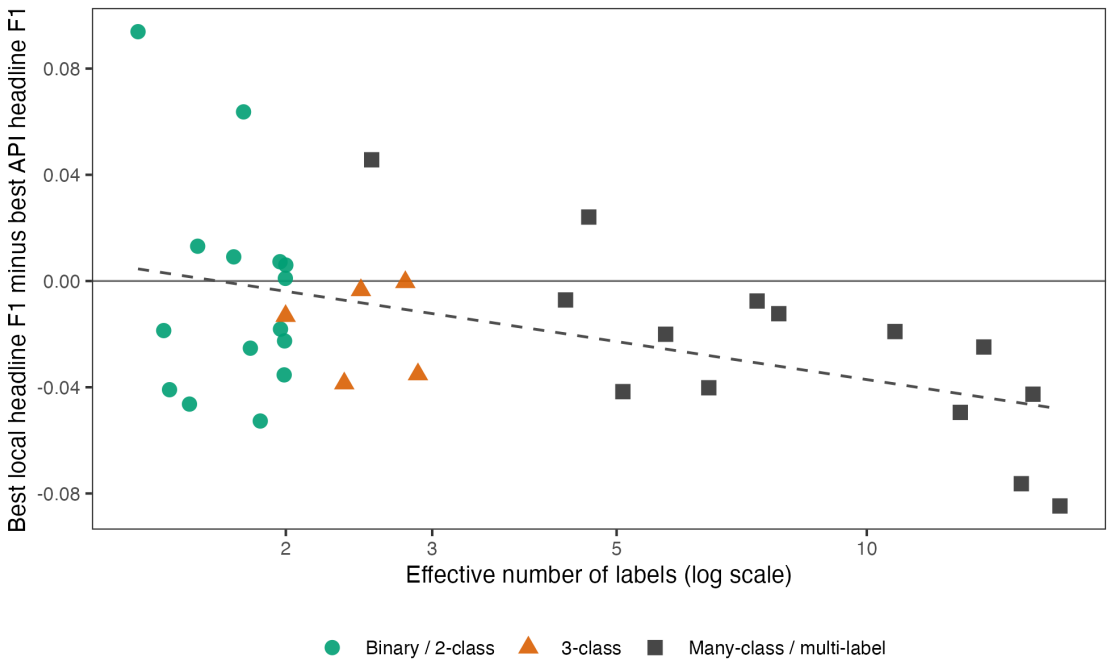

Figure 8: Best local minus best API headline F1 by effective number of labels. The dashed line is a descriptive linear fit across tasks.

Schema or label failure rates above 5 percent

Table 4: Batched cells with schema or label failure rates of at least 5 percent, not a complete list of all task-model cells. Failures include responses that do not parse, responses with the wrong object or array shape, and responses that use labels outside the task schema. Local rows come from the full 34-task b=10 grid for the five local models (170 cells). OpenAI rows come from a limited 10-task prompt-batching run with b=10 and b=20 cells (40 cells). Claude Sonnet and DeepSeek API prompt batching are not covered here.

Task	Model	Batch size	Scope	Schema/label failure rate
ATI topics (7-label)	Gemma 4 26B	10	Local b=10 34-task grid	72%
CCC protest	gpt-5.5	10	Limited OpenAI prompt-batching run	68%
CCC protest type	Qwen3 30B-A3B	10	Local b=10 34-task grid	51%
Wesleyan ads	gpt-5.5	20	Limited OpenAI prompt-batching run	48%
CCC protest	gpt-5.5	20	Limited OpenAI prompt-batching run	40%
BFRS Pakistan	gpt-5.5	20	Limited OpenAI prompt-batching run	32%
Cross-domain topics (8-cl)	Qwen3 30B-A3B	10	Local b=10 34-task grid	30%
BFRS Pakistan	gpt-5.5	10	Limited OpenAI prompt-batching run	28%
Wesleyan ads	gpt-5.5	10	Limited OpenAI prompt-batching run	26%
BFRS violence (12-cl)	Qwen3 30B-A3B	10	Local b=10 34-task grid	18%
Ballard incivility	gpt-5.5	20	Limited OpenAI prompt-batching run	16%
SemEval stance	gpt-5.5	20	Limited OpenAI prompt-batching run	16%
Wesleyan ad tone	Qwen3 30B-A3B	10	Local b=10 34-task grid	16%
BFRS Pakistan	gpt-5.4-nano	10	Limited OpenAI prompt-batching run	12%
SCOTUS sentiment	gpt-5.5	20	Limited OpenAI prompt-batching run	12%
Ballard incivility	gpt-5.5	10	Limited OpenAI prompt-batching run	10%
CMP manifestos	gpt-5.5	10	Limited OpenAI prompt-batching run	10%
GTD attack type (9-cl)	Qwen3 30B-A3B	10	Local b=10 34-task grid	10%
CCC protest	gpt-5.4-nano	20	Limited OpenAI prompt-batching run	8%
BFRS Pakistan	gpt-5.4-nano	20	Limited OpenAI prompt-batching run	8%
PLOVER CAMEO event (18-cl)	Qwen3 30B-A3B	10	Local b=10 34-task grid	6%
CCC protest	gpt-5.4-nano	10	Limited OpenAI prompt-batching run	6%

Performance under alternative metrics

Table 5: Unified 34-task model averages. Coverage is complete for all nine models in this version. Headline F1 is the task-specific summary metric; accuracy and MCC describe overall agreement. For imbalanced tasks, all three answer different questions.

Model	Coverage	Missing	Mean headline F1	Mean accuracy	Mean MCC
Claude Sonnet 4.6	34/34		0.668	0.777	0.632
gpt-5.5	34/34		0.660	0.775	0.627
Gemma 4 31B	34/34		0.648	0.770	0.615
DeepSeek V4 Pro	34/34		0.647	0.772	0.613
Gemma 4 26B	34/34		0.639	0.762	0.597
gpt-5.4-nano	34/34		0.632	0.748	0.591
Mistral Sm 24B	34/34		0.629	0.740	0.583
Qwen3 14B	34/34		0.608	0.749	0.570
Qwen3 30B-A3B	34/34		0.593	0.744	0.555